

UNIVERSITY OF MALAYA

WIA2001 Database Semester 1, Session 25/26 Group Assignment Report

Project Title: Kafe D'Rimba Database System

Occurrence: 6

Group: 11

Lecturer: Prof. Dr. Teh Ying Wah

Name	Matric ID
Xu Jiachen	24061318
Yan Han	24055326
Lee Zi Qi	24061454
Jaad Al Hosaini	24061134
Celine Leong	24067206

Table of Content

1. Introduction	3
2. Business Rules	4
3. System Objectives and Scope	5
3.1 Objectives	5
3.2 Scope	5
4. User Requirements	6
5. Entities and Attributes	7
6. ER Diagram	8
7. Normalization Process	10
8. Implementable ERD	12
9. SQL Statements	13
10. System Testing and Verification	19
11. Screenshots of Tables from the Developed System	27
12. Discussion	30
12.1 Challenges	30
12.2 Lessons Learnt	31
12.3 Future Work	32
13. Appendix	33

1. Introduction

This project involves the design of a database system for **Kafe D'Rimba**, a café located within the University of Malaya. Kafe D'Rimba serves local food and beverages to university students and staff, offering both dine-in and takeaway services. Orders are taken at the counter, where the café owner manages both order placement and payment collection. Food preparation is handled by kitchen staff while beverages are prepared by designated beverage staff.

Currently, the café relies on manual methods such as handwritten records and WhatsApp communication to manage orders, stock updates and daily sales records. As the volume of orders increases, these manual processes have become inefficient and prone to errors, such as missing or duplicated orders, inaccurate sales records and difficulties in tracking popular menu items or staff workload. To address these issues, Kafe D'Rimba requires a structured database system to support daily operations, improve data accuracy, and facilitate better business decision-making.

2. Business Rules

The business rules for Kafe D'Rimba were derived from interviews and observations of the café's daily operations involving the owner and staff members. The following business rules were identified to guide the database design:

1. A customer can place one or more orders but each order must be associated with exactly one customer.
2. Each order is handled by one staff member such as the cashier on duty, while a staff member may handle multiple orders.
3. Each order contains one or more menu items.
4. Menu items have defined attributes including name, category, price and availability status.
5. Menu item names must be unique to prevent confusion in ordering and reporting.
6. Staff members have predefined roles such as Boss, Cashier, or Cook and work in assigned shifts.
7. Orders can be either dine-in or takeaway.

These business rules ensure consistency in operations and serve as the foundation for the database design.

3. System Objective and Scope

3.1 Objectives

- **To digitize and centralize café operations at Kafe D'Rimba**
To replace manual, paper-based records with a centralized database system that stores all café-related data in an organized and accessible manner.
- **To improve the efficiency and accuracy of order handling**
To reduce errors such as missing or duplicated orders by recording customer orders accurately and systematically.
- **To manage menu items and their availability effectively**
To maintain updated menu information, including prices and availability status, so that staff can clearly identify which items can be ordered.
- **To manage staff information and track staff involvement in orders**
To store staff details such as roles and shifts and link each order to the staff member who handled it for better accountability.
- **To record payments accurately for financial tracking**
To capture payment methods and amounts for each order to ensure accurate financial records and support daily sales monitoring.
- **To generate daily sales reports and identify popular menu items**
To provide useful reports that help the café owner analyze sales performance and identify frequently ordered menu items.

3.2 Scope

Included in Scope

- Managing customer orders (order time, type, and total amount)
- Storing and maintaining staff information (role, shift, salary)
- Managing menu items, prices, and availability status
- Recording payment methods and payment amounts
- Generating sales summaries and reports

Excluded from Scope

- Inventory or ingredient stock management
- Online ordering or delivery services

4. User Requirements

Based on the business rules and operational needs, the following user requirements were identified:

a) Menu Management

- Maintain menu items with category and price information.
- Mark menu items as available or unavailable based on stock or preparation capability.

b) Staff Management

- Store staff details including name, role, shift and salary.
- Link each order to the staff member who handled it for accountability and performance tracking.

c) Order Management

- Record customer orders with order time, order type (dine-in or takeaway), and total amount.
- Store ordered menu items along with quantity and unit price.

d) Payment Tracking

- Record payment method such as cash, e-wallet, or card.
- Store the amount paid for each order and ensure that every order has a completed payment record for accurate financial reporting.

e) Reporting

- Generate daily sales summaries automatically.
- Track popular menu items and analyze order trends to support decision-making.

5. Entities and Attributes

From the above business rules and user requirements, the following entities and attributes were identified:

a) Customer

- cust_id (Primary Key)
- name
- phone
- um_student_id (Unique)
- membership_status

b) Staff

- staff_id (Primary Key)
- name
- role
- shift
- salary

c) MenuItem

- item_id (Primary Key)
- name (Unique)
- category
- price
- is_available
- daily_special

d) Order

- order_id (Primary Key)
- order_time
- order_type
- total_amount
- cust_id (Foreign Key)
- staff_id (Foreign Key)

e) OrderDetail

- order_id (Primary Key, Foreign Key)
- item_id (Primary Key, Foreign Key)
- quantity
- unit_price

6. ER Diagram

This section presents the **conceptual Entity Relationship Diagram (ERD)** for the Kafe D'Rimba database system. The ERD identifies the core entities required to support café operations and defines the relationships and cardinalities among them.

Main Entities

- **Customer** (cust_id, name, phone, um_student_id, membership_status)
- **Staff** (staff_id, name, role, shift, salary)
- **MenuItem** (item_id, name, category, price, is_available, daily_special)
- **Order** (order_id, order_time, order_type, total_amount, cust_id, staff_id)
- **OrderDetail** (order_id, item_id, quantity, unit_price)

Key Relationships

- A **Customer** can place one or more **Orders**, and each **Order** is placed by exactly one Customer (**1:M**).
- A **Staff** member can handle multiple **Orders**, and each **Order** is handled by exactly one Staff member (**1:M**).
- An **Order** contains one or more **MenuItems**, and a **MenuItem** can appear in many **Orders** (**M:N**).
This many-to-many relationship is resolved using the associative entity **OrderDetail**, which stores the ordered quantity and the unit price at the time of the order.

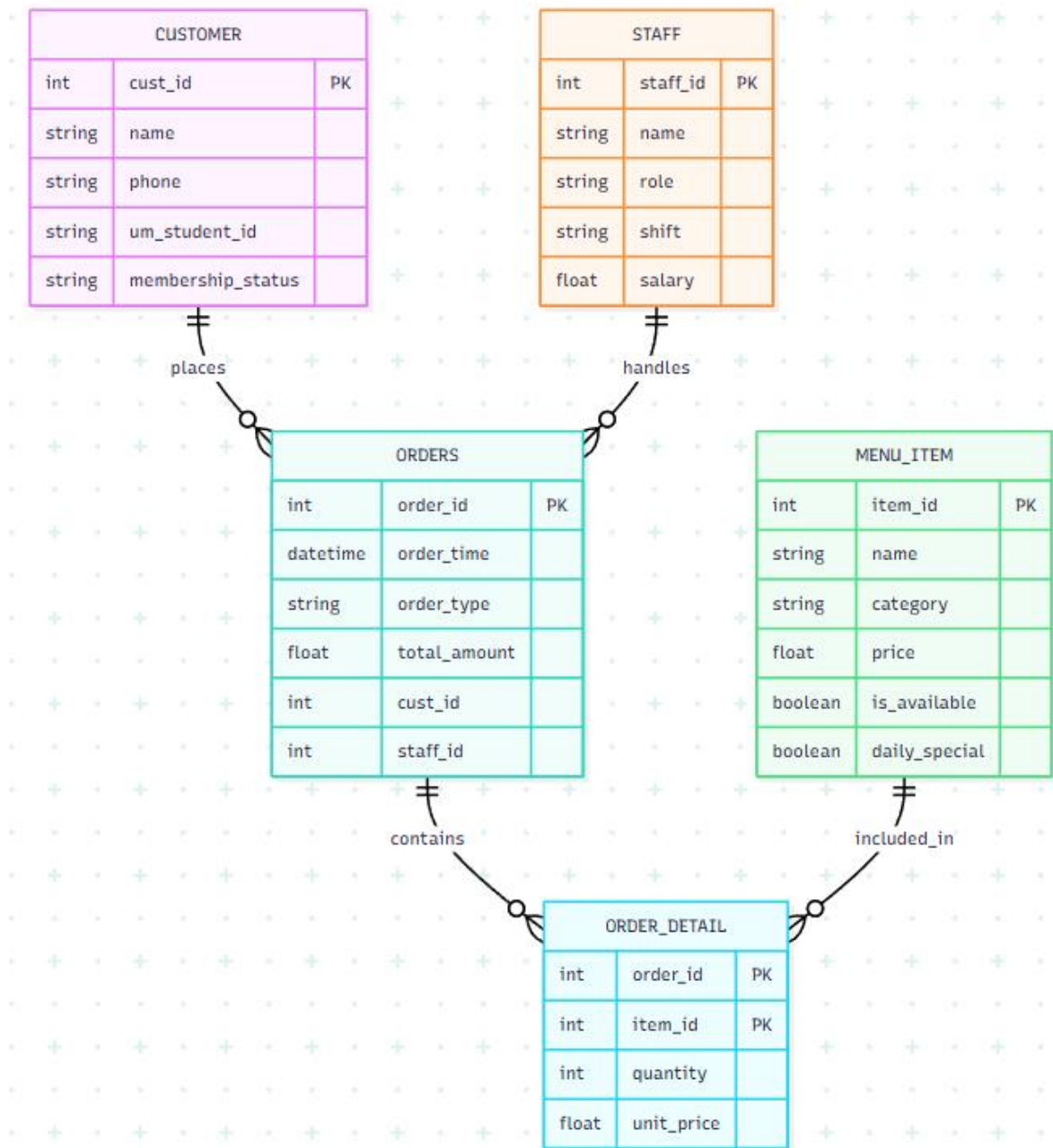


Figure 1: Conceptual Entity Relationship Diagram (ERD) for Kafe D'Rimba Database System

7. Normalization Process

The database design was normalized to **Third Normal Form (3NF)** to minimize redundancy and prevent insertion, update, and deletion anomalies.

Unnormalized Form (UNF)

In the initial unnormalized design, order records could contain repeating groups for menu items (e.g., multiple items and quantities stored within the same order row). This leads to duplicated menu item data and customer/staff information across records, making updates error-prone.

First Normal Form (1NF)

To satisfy 1NF, all attributes were made atomic by separating repeating item groups into a dedicated order-line relation. Each row now represents a single fact.

- **Order**(order_id, order_time, order_type, total_amount, cust_id, staff_id, ...)
- **OrderDetail**(order_id, item_id, quantity, unit_price, ...)

Second Normal Form (2NF)

OrderDetail uses a composite key (**order_id**, **item_id**). Attributes that depend only on part of the composite key must be removed. Menu item descriptive attributes (e.g., name, category, and price) depend only on **item_id**, so they were moved to the **MenuItem** entity to remove partial dependency.

- **MenuItem**(item_id, name, category, price, is_available, daily_special)
- **OrderDetail**(order_id, item_id, quantity, unit_price)

Third Normal Form (3NF)

To satisfy 3NF, transitive dependencies were removed. Customer attributes depend on **cust_id**, and staff attributes depend on **staff_id**, so they were stored in separate entities. The **Order** entity retains only foreign keys to reference **Customer** and **Staff**.

Final 3NF Relations

- **Customer**(cust_id PK, name, phone, um_student_id UNIQUE, membership_status)
- **Staff**(staff_id PK, name, role, shift, salary)
- **MenuItem**(item_id PK, name UNIQUE, category, price, is_available, daily_special)

- **Order**(order_id PK, order_time, order_type, total_amount, cust_id FK, staff_id FK)
- **OrderDetail**(order_id PK/FK, item_id PK/FK, quantity, unit_price)

This final structure satisfies 3NF and improves data integrity, consistency, and maintainability.

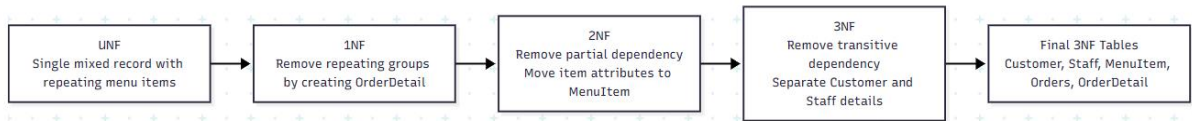


Figure 2: Normalization Overview (UNF to 3NF) for Kafe D'Rimba Database System

8. Implementable ERD

This section presents the **implementable ERD**, which maps the conceptual design into a database-ready schema. It specifies primary keys, foreign keys, and practical constraints required for implementation.

- **Customer** is identified by **cust_id (PK)**. The attribute **um_student_id** is enforced as **UNIQUE**.
- **Staff** is identified by **staff_id (PK)**. Role and shift constraints are applied to restrict values to valid categories.
- **MenuItem** is identified by **item_id (PK)**. The attribute **name** is enforced as **UNIQUE**. Price is constrained to be non-negative, and availability flags are constrained to valid values.
- **Order** is identified by **order_id (PK)**. It includes **cust_id (FK → Customer)** and **staff_id (FK → Staff)**. Order type is constrained to valid values such as dine-in or takeaway.
- **OrderDetail** is an associative entity that implements the many-to-many relationship between **Order** and **MenuItem**. It uses a composite primary key (**order_id, item_id**). Quantity is constrained to be positive, and **unit_price** records the price at the time of ordering.

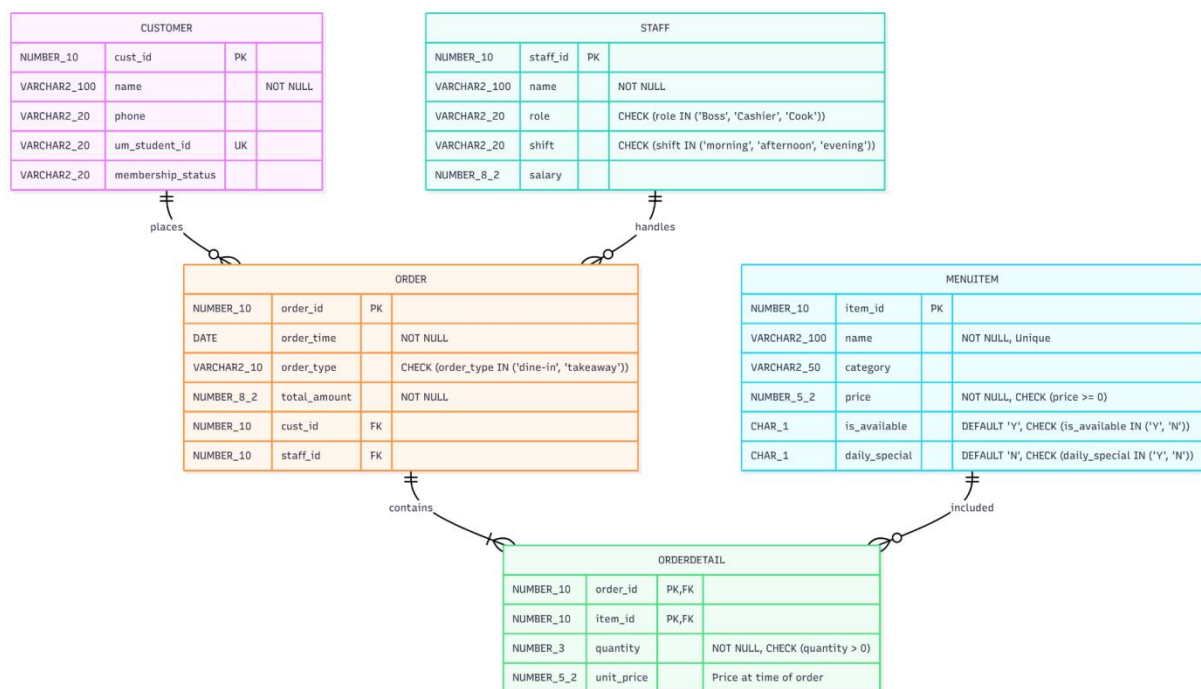


Figure 3: Implementable ERD (Final Normalized ERD) for Kafe D'Rimba Database System

9. SQL statements

DDL Statements:

(A) Create ORDERS table (PK, FK, CHECK)

Purpose: Store order transactions. Each order is linked to exactly one customer and handled by one staff member. Order type is restricted to **dine-in** or **takeaway**.

```
CREATE TABLE ORDERS (  
    ORDER_ID      NUMBER(10,0),  
    ORDER_TIME    DATE,  
    ORDER_TYPE    VARCHAR2(10 BYTE),  
    TOTAL_AMOUNT  NUMBER(8,2),  
    CUST_ID       NUMBER(10,0),  
    STAFF_ID      NUMBER(10,0)  
);  
  
ALTER TABLE ORDERS MODIFY (ORDER_ID NOT NULL ENABLE);  
ALTER TABLE ORDERS MODIFY (ORDER_TIME NOT NULL ENABLE);  
ALTER TABLE ORDERS MODIFY (ORDER_TYPE NOT NULL ENABLE);  
ALTER TABLE ORDERS MODIFY (TOTAL_AMOUNT NOT NULL ENABLE);  
ALTER TABLE ORDERS MODIFY (CUST_ID NOT NULL ENABLE);  
ALTER TABLE ORDERS MODIFY (STAFF_ID NOT NULL ENABLE);  
  
ALTER TABLE ORDERS ADD CONSTRAINT CK_ORDERS_TYPE  
    CHECK (order_type IN ('dine-in','takeaway')) ENABLE;  
  
ALTER TABLE ORDERS ADD CONSTRAINT PK_ORDERS  
    PRIMARY KEY (ORDER_ID) USING INDEX ENABLE;  
  
ALTER TABLE ORDERS ADD CONSTRAINT FK_ORDERS_CUSTOMER  
    FOREIGN KEY (CUST_ID) REFERENCES CUSTOMER (CUST_ID)  
    ENABLE;  
  
ALTER TABLE ORDERS ADD CONSTRAINT FK_ORDERS_STAFF  
    FOREIGN KEY (STAFF_ID) REFERENCES STAFF (STAFF_ID) ENABLE;
```

Expected outcome: ORDERS is created with valid constraints. Invalid order types are rejected; orders must reference existing customers and staff.

(B) Create ORDERDETAIL table (Composite PK + FK + CHECK)

Purpose: Store menu items within each order. Composite primary key prevents duplicate items in the same order and quantity must be > 0.

```

CREATE TABLE ORDERDETAIL (
    ORDER_ID    NUMBER(10,0) NOT NULL,
    ITEM_ID     NUMBER(10,0) NOT NULL,
    QUANTITY    NUMBER(3,0)  NOT NULL,
    UNIT_PRICE  NUMBER(5,2),
    CONSTRAINT PK_ORDERDETAIL PRIMARY KEY (ORDER_ID, ITEM_ID),
    CONSTRAINT CK_OD_QTY CHECK (QUANTITY > 0),
    CONSTRAINT FK_OD_ORDER FOREIGN KEY (ORDER_ID) REFERENCES
ORDERS (ORDER_ID),
    CONSTRAINT FK_OD_ITEM  FOREIGN KEY (ITEM_ID)  REFERENCES
MENUITEM (ITEM_ID)
);

```

(C) Indexes (Performance)

Purpose: Improve query performance for frequent searches by customer/staff/item.

```

CREATE INDEX IDX_ORDERS_CUST  ON ORDERS (CUST_ID);
CREATE INDEX IDX_ORDERS_STAFF ON ORDERS (STAFF_ID);
CREATE INDEX IDX_OD_ITEM      ON ORDERDETAIL (ITEM_ID);

```

(D) Auto-generated ID (Sequence + Trigger)

Purpose: Automatically generate primary key values without manual input.

```

CREATE SEQUENCE SEQ_ORDERS START WITH 5003 INCREMENT BY 1 NOCACHE
NOCYCLE;

CREATE OR REPLACE TRIGGER TRG_ORDERS_BI
BEFORE INSERT ON ORDERS
FOR EACH ROW
BEGIN
    IF :NEW.order_id IS NULL THEN
        :NEW.order_id := SEQ_ORDERS.NEXTVAL;
    END IF;
END;
/

```

(E) Views (Reporting / Receipt)

Purpose: Provide reusable queries for daily sales reporting and order receipts.

```

CREATE OR REPLACE VIEW V_DAILY_SALES AS
SELECT TRUNC(order_time) AS sales_date,
       COUNT(*) AS total_orders,
       SUM(total_amount) AS total_revenue
FROM ORDERS
GROUP BY TRUNC(order_time);

CREATE OR REPLACE VIEW V_ORDER_RECEIPT AS
SELECT o.order_id, o.order_time, o.order_type,
       c.name AS customer_name,
       s.name AS staff_name,
       m.name AS item_name,
       od.quantity, od.unit_price,
       (od.quantity * od.unit_price) AS line_total,
       o.total_amount
FROM ORDERS o
JOIN CUSTOMER c   ON c.cust_id = o.cust_id
JOIN STAFF s      ON s.staff_id = o.staff_id
JOIN ORDERDETAIL od ON od.order_id = o.order_id
JOIN MENUITEM m   ON m.item_id = od.item_id;

```

DML Statements

(A) Insert a new order + order items (Transaction)

Purpose: Demonstrate creating an order and inserting its items in one transaction.

ORDER_ID is auto-generated by trigger.

```

DECLARE
    v_order_id NUMBER;
BEGIN
    INSERT INTO ORDERS (order_time, order_type, total_amount,
                       cust_id, staff_id)
    VALUES (SYSDATE, 'dine-in', 15.50, 1, 100)
    RETURNING order_id INTO v_order_id;

    INSERT INTO ORDERDETAIL (order_id, item_id, quantity,
                           unit_price)
    VALUES (v_order_id, 1000, 2, 6.50);

    INSERT INTO ORDERDETAIL (order_id, item_id, quantity,
                           unit_price)
    VALUES (v_order_id, 1001, 1, 2.50);
END;
/

```

(B) Update menu item availability

Purpose: Mark an item as sold out.

```
UPDATE MENUITEM
SET is_available = 'N'
WHERE item_id = 1000;
```

(C) Delete with integrity consideration

Purpose: Demonstrate referential integrity (cannot delete referenced parent rows).

```
DELETE FROM CUSTOMER
WHERE cust_id = 1;
```

Expected outcome: If CUSTOMER has orders, deletion will fail due to FK constraint, proving data integrity enforcement.

Complex Queries (Reports & Search)

This section presents complex SQL queries that support key system functions such as receipt generation, sales reporting, identifying top-selling items, and staff/customer performance analysis. The queries demonstrate multi-table JOIN operations, aggregate functions (COUNT/SUM), GROUP BY/HAVING clauses, and nested/analytic queries for decision-making reports.

Q1. Order Receipt:

Purpose: Generate a complete receipt for a selected order, including customer, staff, item breakdown and line totals.

```
SELECT o.order_id,
       o.order_time,
       o.order_type,
       c.name AS customer_name,
       s.name AS staff_name,
       m.name AS item_name,
       od.quantity,
       od.unit_price,
       (od.quantity * od.unit_price) AS
line_total,
       o.total_amount
FROM orders o
JOIN customer c      ON c.cust_id = o.cust_id
JOIN staff s         ON s.staff_id = o.staff_id
JOIN orderdetail od  ON od.order_id = o.order_id
JOIN menuitem m      ON m.item_id = od.item_id
WHERE o.order_id = :p_order_id
ORDER BY m.name;
```


Q2. Daily Sales Report (COUNT + SUM + GROUP BY)

Purpose: Summarize total orders and total revenue per day for management reporting.

```
SELECT TRUNC(order_time) AS sales_date,
       COUNT(*) AS total_orders,
       SUM(total_amount) AS total_revenue
FROM orders
GROUP BY TRUNC(order_time)
ORDER BY sales_date;
```

Q3. Top-selling Menu Items (JOIN + SUM + RANK)

Purpose: Identify best-selling items by quantity and sales amount.

```
SELECT m.item_id,
       m.name AS item_name,
       m.category,
       SUM(od.quantity) AS total_qty_sold,
       SUM(od.quantity * od.unit_price) AS total_sales
FROM orderdetail od
JOIN menuitem m ON m.item_id = od.item_id
GROUP BY m.item_id, m.name, m.category
ORDER BY total_sales DESC;
```

Q4. Staff Performance Ranking

Purpose: Rank staff by total revenue handled (performance report).

```
SELECT s.staff_id,
       s.name AS staff_name,
       s.role,
       COUNT(o.order_id) AS orders_handled,
       SUM(o.total_amount) AS total_revenue,
       RANK() OVER (ORDER BY SUM(o.total_amount) DESC)
AS revenue_rank
FROM staff s
JOIN orders o ON o.staff_id = s.staff_id
GROUP BY s.staff_id, s.name, s.role
ORDER BY total_revenue DESC;
```

Q5. Customer Order History

Purpose: Display a customer's order history with total spent.

```

SELECT c.cust_id,
       c.name AS customer_name,
       o.order_id,
       o.order_time,
       o.order_type,
       o.total_amount
FROM customer c
JOIN orders o ON o.cust_id = c.cust_id
WHERE c.cust_id = :p_cust_id
ORDER BY o.order_time DESC;

```

Q7. Identify Orders with Total Mismatch

Purpose: Verify that ORDERS.total_amount matches the sum of its order lines.

```

SELECT o.order_id,
       o.total_amount AS orders_total,
       SUM(od.quantity * od.unit_price) AS
detail_total,
       (o.total_amount - SUM(od.quantity *
od.unit_price)) AS difference
FROM orders o
JOIN orderdetail od ON od.order_id = o.order_id
GROUP BY o.order_id, o.total_amount
HAVING o.total_amount <> SUM(od.quantity *
od.unit_price);

```

10. System Testing and Verification

1. Testing Approach

The system was tested within the **Oracle SQL Developer** environment to verify that the database implementation aligns with business rules and data integrity requirements. The testing process is categorized into three primary areas:

1. **Structural Testing:** Validates the successful creation of database objects, including tables, primary and foreign keys, *CHECK* constraints, *UNIQUE* constraints, indexes, and views.
2. **Constraint Testing:** Conducted through "negative testing" (deliberately entering invalid data) to ensure constraints effectively block illegal transactions, such as invalid order types, duplicate menu names, or missing foreign key references.
3. **Functional Query Testing:** Executes complex queries and views (e.g., receipts, daily sales reports, and top-selling items) to verify the accuracy of the reporting and search functionalities.

T1 — Verify tables were created successfully

The screenshot shows the Oracle SQL Developer interface. The top pane contains the following SQL query:

```
SELECT table_name
FROM user_tables
WHERE table_name IN ('CUSTOMER', 'STAFF', 'MENUITEM', 'ORDERS', 'ORDERDETAIL');
```

The bottom pane, titled "Script Output", shows the results of the query. It indicates "Task completed in 0.268 seconds" and displays a table with the following data:

TABLE_NAME
CUSTOMER
MENUITEM
ORDERDETAIL
ORDERS
STAFF

T2 — Verify primary keys exist

The screenshot shows the Oracle SQL Developer interface. The top pane contains the following SQL query:

```
SELECT constraint_name, table_name, constraint_type
FROM user_constraints
WHERE table_name IN ('CUSTOMER', 'STAFF', 'MENUITEM', 'ORDERS', 'ORDERDETAIL')
AND constraint_type = 'P'
ORDER BY table_name;
```

The bottom pane, titled "Script Output", shows the results of the query. It indicates "Task completed in 0.822 seconds" and displays a table with the following data:

CONSTRAINT_NAME	TABLE_NAME
PK_CUSTOMER	CUSTOMER
PK_MENUITEM	MENUITEM
PK_ORDERDETAIL	ORDERDETAIL
PK_ORDERS	ORDERS

T3 — Verify foreign keys exist

```
SELECT constraint_name, table_name, r_constraint_name
FROM user_constraints
WHERE constraint_type = 'R'
AND table_name IN ('ORDERS', 'ORDERDETAIL')
ORDER BY table_name;
```

Script Output x | Task completed in 0.939 seconds

CONSTRAINT_NAME	TABLE_NAME
FK_OD_ORDER	ORDERDETAIL
FK_OD_ITEM	ORDERDETAIL
FK_ORDERS_CUSTOMER	ORDERS
FK_ORDERS_STAFF	ORDERS

T4 — Trigger/Sequence auto-generate ID

```
INSERT INTO CUSTOMER (name, phone, um_student_id, membership_status)
VALUES ('AutoID_Test', '+601111111111', 'UM00000001', 'Active');

SELECT cust_id, name, um_student_id
FROM CUSTOMER
WHERE um_student_id = 'UM00000001';
```

Script Output x | Task completed in 0.142 seconds

1 row inserted.

CUST_ID	NAME	UM_STUDENT_ID
8	AutoID_Test	UM00000001

Note: We didn't provide any ID information in the code, but the output automatically generated one.

T5 — UNIQUE constraint on MENUITEM.NAME

```
INSERT INTO MENUITEM (name, category, price, is_available, daily_special)
VALUES ('Latte', 'Drink', 7.00, 'Y', 'N');
```

Script Output x | Task completed in 0.326 seconds

Error starting at line : 12 in command -
INSERT INTO MENUITEM (name, category, price, is_available, daily_special)
VALUES ('Latte', 'Drink', 7.00, 'Y', 'N')
Error report -
ORA-00001: unique constraint (SYSTEM.UQ_MENUITEM_NAME) violated

<https://docs.oracle.com/error-help/db/ora-00001/>

More Details :
<https://docs.oracle.com/error-help/db/ora-00001/>

Explanation:

The insertion failed with an *ORA-00001* error, verifying that the UNIQUE constraint on *MENUITEM.NAME* successfully prevents duplicate menu item names.

T6 — UNIQUE constraint on CUSTOMER.UM_STUDENT_ID

```
INSERT INTO CUSTOMER (name, phone, um_student_id, membership_status)
VALUES ('DuplicateUM', '+60122222222', 'UM1234567', 'Active');
```

Script Output x

Task completed in 0.022 seconds

Error starting at line : 7 in command -
INSERT INTO CUSTOMER (name, phone, um_student_id, membership_status)
VALUES ('DuplicateUM', '+60122222222', 'UM1234567', 'Active')
Error report -
ORA-00001: unique constraint (SYSTEM.UQ_CUSTOMER_UM) violated

<https://docs.oracle.com/error-help/db/ora-00001/>

More Details :
<https://docs.oracle.com/error-help/db/ora-00001/>

Explanation:

The *ORA-00001* error confirms that duplicate *UM_STUDENT_ID* values are not allowed in the *CUSTOMER* table.

T7 — CHECK constraint on ORDERS.ORDER_TYPE

```
INSERT INTO ORDERS (order_time, order_type, total_amount, cust_id, staff_id)
VALUES (SYSDATE, 'delivery', 10.00, 1, 100);
```

Script Output x

Task completed in 0.044 seconds

Error starting at line : 7 in command -
INSERT INTO ORDERS (order_time, order_type, total_amount, cust_id, staff_id)
VALUES (SYSDATE, 'delivery', 10.00, 1, 100)
Error report -
ORA-02290: check constraint (SYSTEM.CK_ORDERS_TYPE) violated

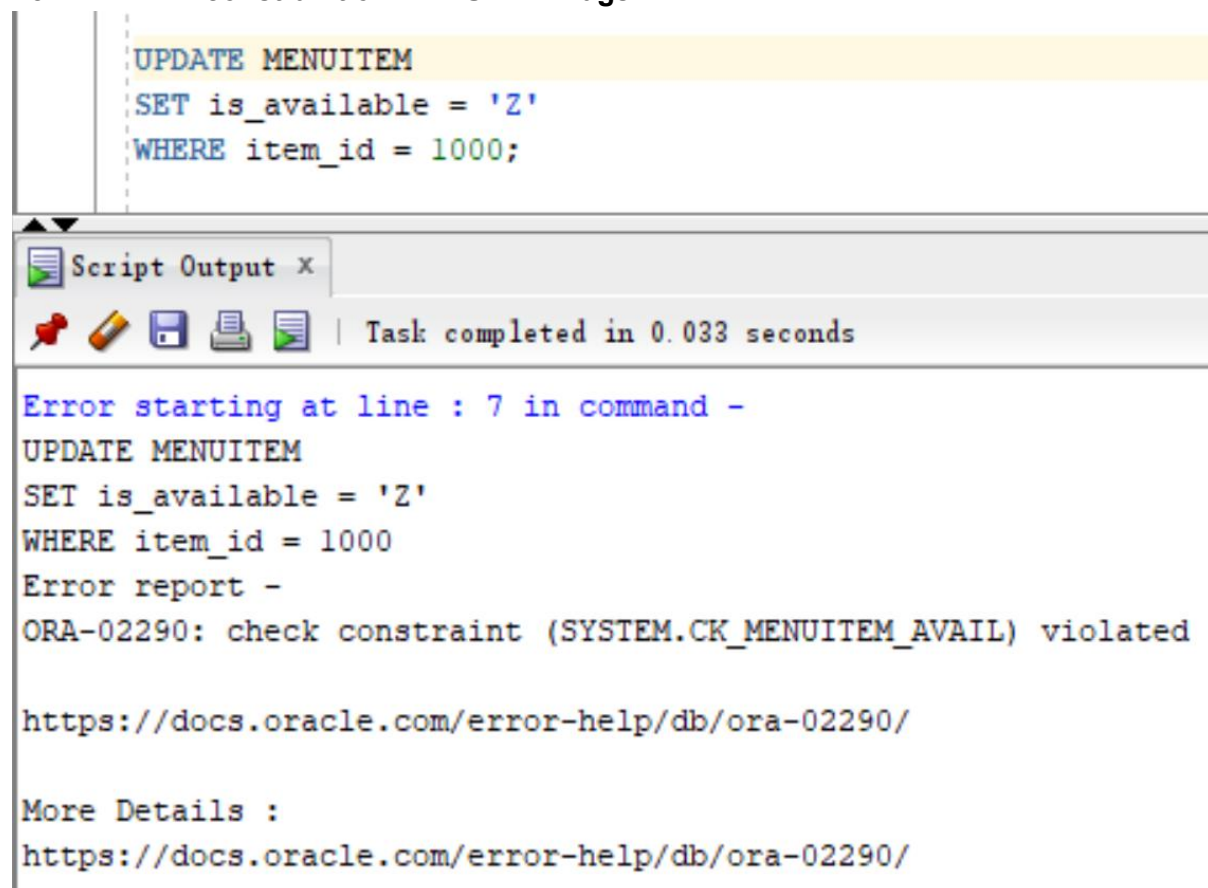
<https://docs.oracle.com/error-help/db/ora-02290/>

More Details :
<https://docs.oracle.com/error-help/db/ora-02290/>

Explanation:

The *ORA-02290* error confirms that only valid order types (*dine-in* and *takeaway*) are allowed in the *ORDERS* table.

T8 — CHECK constraint on MENUITEM flags



The screenshot shows a database script execution window. At the top, a SQL script is displayed: `UPDATE MENUITEM`, `SET is_available = 'Z'`, and `WHERE item_id = 1000;`. Below the script, a status bar indicates "Task completed in 0.033 seconds". The main area of the window displays an error message: "Error starting at line : 7 in command -", followed by the same SQL script, and then "Error report -", "ORA-02290: check constraint (SYSTEM.CK_MENUITEM_AVAIL) violated", and two links to Oracle documentation: <https://docs.oracle.com/error-help/db/ora-02290/> and "More Details : https://docs.oracle.com/error-help/db/ora-02290/".

```
UPDATE MENUITEM
SET is_available = 'Z'
WHERE item_id = 1000;

Script Output x
Task completed in 0.033 seconds

Error starting at line : 7 in command -
UPDATE MENUITEM
SET is_available = 'Z'
WHERE item_id = 1000
Error report -
ORA-02290: check constraint (SYSTEM.CK_MENUITEM_AVAIL) violated

https://docs.oracle.com/error-help/db/ora-02290/

More Details :
https://docs.oracle.com/error-help/db/ora-02290/
```

Explanation:

The *ORA-02290* error confirms that only valid values ('Y' or 'N') are allowed for the *IS_AVAILABLE* flag in the *MENUITEM* table.

T9 — CHECK constraint on ORDERDETAIL.QUANTITY > 0

```
INSERT INTO ORDERDETAIL (order_id, item_id, quantity, unit_price)
VALUES (5000, 1000, 0, 6.50);
```

Script Output x

Task completed in 0.021 seconds

Error starting at line : 7 in command -

```
INSERT INTO ORDERDETAIL (order_id, item_id, quantity, unit_price)
VALUES (5000, 1000, 0, 6.50)
```

Error report -

ORA-02290: check constraint (SYSTEM.CK_OD_QTY) violated

<https://docs.oracle.com/error-help/db/ora-02290/>

More Details :

<https://docs.oracle.com/error-help/db/ora-02290/>

Explanation:

This test attempts to insert an order detail record with a quantity value of 0. The database correctly rejects the insertion and returns an **ORA-02290** error, confirming that the CHECK constraint on **ORDERDETAIL.QUANTITY** enforces the rule that item quantity must be greater than zero.

T10 — FK constraint: ORDERS must reference existing CUSTOMER

```
INSERT INTO ORDERS (order_time, order_type, total_amount, cust_id, staff_id)
VALUES (SYSDATE, 'dine-in', 10.00, 9999, 100);
```

Script Output x

Task completed in 0.029 seconds

Error starting at line : 7 in command -

```
INSERT INTO ORDERS (order_time, order_type, total_amount, cust_id, staff_id)
VALUES (SYSDATE, 'dine-in', 10.00, 9999, 100)
```

Error report -

ORA-02291: integrity constraint (SYSTEM.FK_ORDERS_CUSTOMER) violated - parent key not found

<https://docs.oracle.com/error-help/db/ora-02291/>

More Details :

<https://docs.oracle.com/error-help/db/ora-02291/>

Explanation:

This test attempts to insert an order that references a non-existent customer ID. The

database correctly rejects the insertion and returns an *ORA-02291* error, confirming that the foreign key constraint on *ORDERS.CUST_ID* enforces referential integrity by ensuring that every order is linked to an existing customer.

T11 — FK constraint: ORDERDETAIL must reference existing ORDERS

```
INSERT INTO ORDERDETAIL (order_id, item_id, quantity, unit_price)
VALUES (9999, 1000, 1, 6.50);
```

Script Output x | Task completed in 0.022 seconds

```
Error starting at line : 7 in command -
INSERT INTO ORDERDETAIL (order_id, item_id, quantity, unit_price)
VALUES (9999, 1000, 1, 6.50)
Error report -
ORA-02291: integrity constraint (SYSTEM.FK_OD_ORDER) violated - parent key not found

https://docs.oracle.com/error-help/db/ora-02291/

More Details :
https://docs.oracle.com/error-help/db/ora-02291/
```

Explanation:

The *ORA-02291* error confirms that an order detail cannot exist without a corresponding order record.

T12 — Referential integrity on DELETE

```
DELETE FROM CUSTOMER
WHERE cust_id = 1;
```

Script Output x | Task completed in 0.028 seconds

```
Error starting at line : 7 in command -
DELETE FROM CUSTOMER
WHERE cust_id = 1
Error report -
ORA-02292: integrity constraint (SYSTEM.FK_ORDERS_CUSTOMER) violated - child record found

https://docs.oracle.com/error-help/db/ora-02292/

More Details :
https://docs.oracle.com/error-help/db/ora-02292/
```

Explanation:

The **ORA-02292** error confirms that a customer cannot be deleted while related order records still exist.

Verification of Views / Reports

T13 — Verify V_DAILY_SALES output

```
SELECT *
FROM V_DAILY_SALES
ORDER BY sales_date;
```

Script Output x		
Task completed in 0.069		
SALES_DAT	TOTAL_ORDERS	TOTAL_REVENUE
01-JAN-26	1	15.5
02-JAN-26	1	19
03-JAN-26	1	20.5

T14 — Verify V_ORDER_RECEIPT (receipt breakdown)(receipt view)

```
SELECT order_id, customer_name, staff_name, item_name,
       quantity, unit_price, line_total, total_amount
FROM V_ORDER_RECEIPT
WHERE order_id = 5001
ORDER BY item_name;
```

Script Output x	
Task completed in 0.089 seconds	
ORDER_ID CUSTOMER_NAME	STAFF_NAME
5001 Ali	Aina
5001 Ali	Aina

Data Integrity Cross-check

T15 — Verify order total matches sum of order lines

Test Objective

To verify that the **TOTAL_AMOUNT** stored in the **ORDERS** table is consistent with the sum of item-level amounts (**QUANTITY × UNIT_PRICE**) recorded in the **ORDERDETAIL** table.

```
SELECT o.order_id,  
       o.total_amount AS orders_total,  
       SUM(od.quantity * od.unit_price) AS detail_total,  
       (o.total_amount - SUM(od.quantity * od.unit_price)) AS difference  
FROM ORDERS o  
JOIN ORDERDETAIL od ON od.order_id = o.order_id  
GROUP BY o.order_id, o.total_amount  
HAVING o.total_amount <> SUM(od.quantity * od.unit_price);
```

Script Output x

Task completed in 0.023 seconds

no rows selected

The query returned no rows, indicating that for all orders in the database, the *TOTAL_AMOUNT* value stored in the *ORDERS* table is exactly equal to the sum of *QUANTITY* × *UNIT_PRICE* from the corresponding records in the *ORDERDETAIL* table. This confirms that the order totals are consistent and that no data integrity issues exist between order headers and order details.

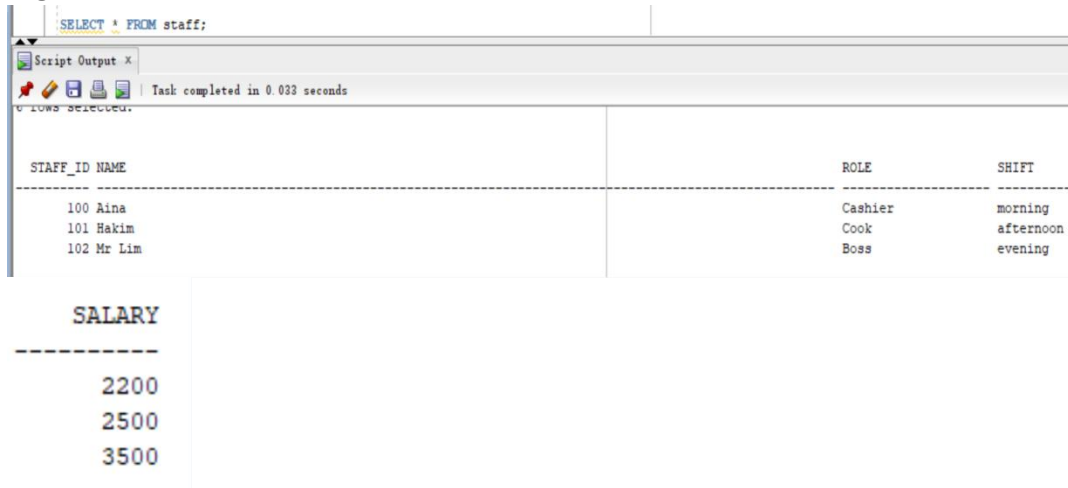
11. Screenshots of tables from the developed system

Figure 1 — CUSTOMER Table



CUST_ID	NAME	PHONE	UM_STUDENT_ID	MEMBERSHIP_STATUS
1	YH	+60123456789	UM1234567	Active
2	Ali	+60111222333	UM7654321	Inactiv
3	Siti	+60199887766	UM2345678	Active
4	Farah	+60147778899	UM4567890	Active
5	Test AutoID	+60100000000	UM9999999	Active
8	AutoID_Test	+60111111111	UM0000001	Active

Figure 2 — STAFF Table

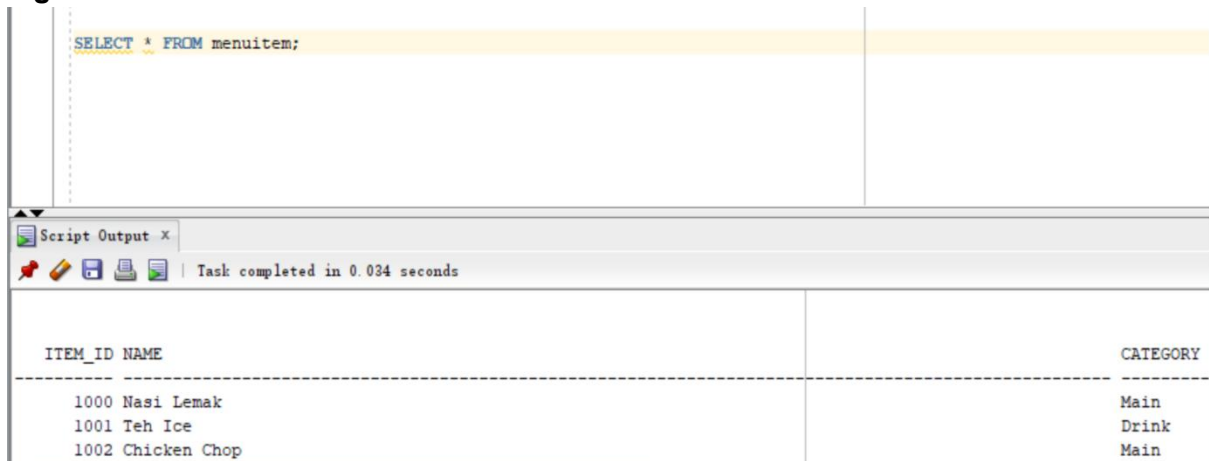


STAFF_ID	NAME	ROLE	SHIFT
100	Aina	Cashier	morning
101	Hakim	Cook	afternoon
102	Mr Lim	Boss	evening

SALARY

2200
2500
3500

Figure 3 — MENUITEM Table



ITEM_ID	NAME	CATEGORY
1000	Nasi Lemak	Main
1001	Teh Ice	Drink
1002	Chicken Chop	Main

Figure 4 — ORDERS Table

`SELECT * FROM orders`

Script Output x

Task completed in 0.023 seconds

ORDER_ID	ORDER_TIM	ORDER_TYPE	TOTAL_AMOUNT	CUST_ID	STAFF_ID
5000	01-JAN-26	dine-in	15.5	1	100
5001	02-JAN-26	takeaway	19	2	100
5002	03-JAN-26	dine-in	20.5	3	101

Figure 5 — ORDERDETAIL Table

`SELECT * FROM orderdetail`

Script Output x

Task completed in 0.02 seconds

ORDER_ID	ITEM_ID	QUANTITY	UNIT_PRICE
5000	1000	2	6.5
5000	1001	1	2.5
5001	1002	1	12
5001	1003	1	7
5002	1000	1	6.5
5002	1003	2	7

Figure 6 — V_ORDER_RECEIPT View Output

```
SELECT item_name,
       quantity,
       unit_price,
       line_total,
       total_amount
FROM V_ORDER_RECEIPT
WHERE order_id = 5000;
```

Script Output x

Task completed in 0.019 seconds

6 rows selected.

ITEM_NAME	QUANTITY	UNIT_PRICE	LINE_TOTAL	TOTAL_AMOUNT
Nasi Lemak	2	6.5	13	15.5
Teh Ice	1	2.5	2.5	15.5

Figure X — V_DAILY_SALES View Output

```
select * from V_DAILY_SALES
```

Script Output x

Task completed in 0.023 second

SALES_DAT	TOTAL_ORDERS	TOTAL_REVENUE
01-JAN-26	1	15.5
02-JAN-26	1	19
03-JAN-26	1	20.5

12. Discussion

12.1 Challenges

During the development of the Kafe D'Rimba database system, the group faced several challenges related to both data collection and database implementation. One of the initial challenges occurred during the data collection stage, where several organizations were contacted through email to request interviews and requirement information. However, most of the organizations did not respond, which delayed the requirement gathering process. Eventually, Kafe D'Rimba responded and agreed to participate in the interview, allowing the project to proceed. This experience required the group to adjust the project timeline and highlighted the difficulty of obtaining real-world data from external organizations.

Another major challenge was converting the café's daily operations into a structured database design. Identifying the correct entities, attributes, and relationships required careful discussion to ensure the system reflected real business processes. Handling the many-to-many relationship between orders and menu items using the ORDERDETAIL table was initially difficult but necessary to apply normalization and reduce data duplication.

The group also faced challenges while working with Oracle SQL. Defining primary keys, foreign keys, and CHECK constraints required careful attention to Oracle-specific syntax. Creating tables in the correct order to avoid dependency errors was another difficulty. Writing complex SQL queries involving multiple tables and aggregate functions was challenging at first and required repeated testing.

12.2 Lessons learnt

Through the Kafe D'Rimba database project, the group learned several important lessons related to planning and teamwork. One key lesson was the importance of good planning and time management. Clearly dividing tasks among group members and setting realistic deadlines helped the team stay organized and complete each stage of the project smoothly. Regular communication among members also ensured that everyone understood their roles and the project goals, which improved teamwork and coordination.

The project also showed us the importance of being flexible and solving problems effectively. During the development process, we faced technical challenges and unexpected issues that required changes to our original plan. These situations taught us to stay calm, think critically, and focus on finding solutions. As a result, we became more confident in handling challenges and working under pressure.

In addition, the project helped strengthen our technical skills in database design and implementation. We gained hands-on experience in applying database concepts such as normalization, defining constraints, and writing SQL queries. The importance of proper documentation and testing was also reinforced, as keeping organized records and checking system functionality helped ensure accuracy and reliability. Overall, this project provided valuable experience that will be useful for future academic work and real-world applications.

12.3 Future work

For future improvements, the Kafe D'Rimba database system can be enhanced by integrating an inventory management module. This feature would allow the café to track ingredient stock levels in real time, monitor usage based on orders, and receive alerts when stock levels are low. Such an enhancement would help reduce food wastage, prevent stock shortages, and support better purchasing and restocking decisions. In addition, the system could be extended to include supplier information and purchase records to further improve inventory control and operational efficiency.

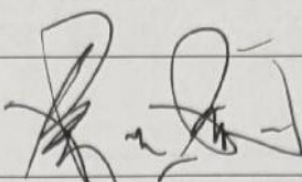
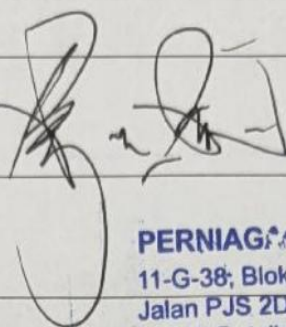
Other possible enhancements include implementing user authentication and role-based access control to strengthen system security. By assigning different access levels to managers, cashiers, and kitchen staff, the system can better protect sensitive data and prevent unauthorized changes. Furthermore, visual reporting dashboards can be added to present sales data, popular menu items and order trends in an easy-to-understand graphical format. These improvements would provide clearer insights for decision-making and make the database system more effective and scalable for long-term café operations.

13. Appendix

Appendix A

Interview Acknowledgement Form

Information	Details
Date of Interview:	1.Nov.2025
Interviewed by (Students):	1.Xu Jiachen 24061318 2.Yan Han 24055326 3.Lee Zi Qi 24061454 4.Jaad Al Hosaini 24061134 5.Celine Leong 24067206
Organization Name:	Kafe D'Rimba
Representative Name:	Mr. Mohd Zin Idris
Position / Role:	Café Manager
Signature:	See attached image below
Official Stamp (if any):	Attached below

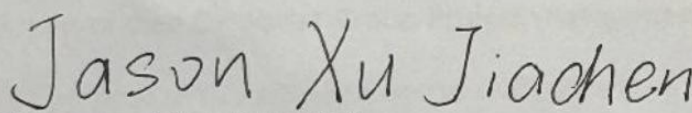
Information	Details
Representative Name:	
Position / Role:	
Signature:	
Official Stamp (if any):	 PERNIAGAAN SPN SITI (002755675-A) 11-G-38; Blok II, Desa Mentari 3, Jalan PJS 2D/1, Taman Medan, 46000 Petaling Jaya, Selangor D.E.

We sincerely thank you for your time and cooperation.

Your support has greatly contributed to our understanding of real-world business operations and database requirements.

Verified by:

Xu Jiachen(24061318)


(Signature of Group Leader)

Date: 1. Nov. 2025

Faculty of Computer Science & Information Technology (FCSIT)
University of Malaya, Kuala Lumpur, Malaysia

Figure: Interview acknowledgement form with signature and official stamp of Cafe' Rimba



Figure: Photo of group members at Kafe D'Rimba

Appendix B

Group Working Documents and Supporting Materials

The Google Drive folder was used solely for internal group coordination and documentation purposes. Selected outputs from these materials have been consolidated and presented in the main report.

Google Drive link (for reference):

<https://drive.google.com/drive/folders/12sBMYScfCFaIL4abaulozppoEXdJUvE>